ASE CHAPTER 4

T2 Descriptive & Process Exercises Pack · Software Project Planning

Section 1: Core Textbook Descriptive Exercises (10 Items)

Exercise 4.1. Activities During Software Project Planning

What are various activities during software project planning?

Software project planning is an umbrella management process that begins once the Software Requirements Specification (SRS) is finalized. It encompasses five core interdependent activities:

- **1. Software Scope Definition:** Establishes the project boundaries, primary functional objectives, external interfaces, and performance constraints derived from the SRS.
- **2. Software Estimation (Size, Effort, Cost, and Time):** Determines the size of the deliverable (using LOC or Function Points), calculates required development effort in Person-Months (PM), predicts total project costs (hardware, software, personnel), and establishes realistic schedule milestones.
- **3. Resource Allocation and Staffing:** Identifies the human skills, organizational roles, development hardware, system software tools, and physical facilities necessary to execute the project over time.
- **4. Software Risk Analysis and Management:** Proactively identifies potential technical, operational, and managerial pitfalls, evaluates their likelihood and impact (Risk Exposure), and creates mitigation, monitoring, and contingency strategies.
- **5. Project Tracking and Control Mechanisms:** Formulates a concrete Project Management Plan (PMP) with work breakdown structures (WBS), Gantt charts, milestone reviews, and quality gates to monitor actual versus planned progress.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.1, pp. 139–141; Companion Slides 4–5.

Exercise 4.2. Software Size Estimation Techniques

Describe any two software size estimation techniques.

Two software-size estimation techniques are LOC counting and function counting.

- **Lines of Code (LOC):** estimates size by counting source-code lines or statements. It depends on the chosen language and measures implementation.
- **Function Count / Function Points:** identifies five functional-unit types—EI, EO, EQ, ILF, and EIF. Their counts are classified by complexity and weighted to calculate UFP; the CAF can then adjust UFP to FP.

> Function counting supplies the functional-unit counts used in the function-point calculation; the terms are related but should not be treated as identical.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.3.1–§4.3.2, pp. 143–150; Companion Slides 6–32.*

Exercise 4.6. Concept and Acceptance of Function Points

Explain the concept of function points. Why are FPs becoming acceptable in industry?

> **Function Points estimate software size from the functions visible to the user.** The calculation counts five functional-unit types: External Inputs, External Outputs, External Inquiries, Internal Logical Files, and External Interface Files. Complexity weights produce UFP, which may be adjusted by the 14 General System Characteristics.
>
> **Reasons given for using Function Points include:**
>
> - They are linked to the requirements, so a requirements change can be followed by a re-estimate.
> - They describe the system from the user's external view and can be understood by non-technical users.
> - They avoid some inconsistencies that arise when LOC is used as the size measure.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.3.2, pp. 145–150; Companion Slides 10, 31.*

Exercise 4.9. Albrecht's Function Count Method with a Worked Example

Describe the Albrecht's function count method with a suitable example.

> **Albrecht's Function Count and its Function-Point Calculation: The method first counts and weights five functional-unit types to obtain UFP, then applies the CAF to obtain Function Points.**
>
> - **1. Phase 1: Computing Unadjusted Function Points (UFP):** The system's information domain is audited across five component types, and each instance is weighted by its complexity level:
> - **External Inputs (EI):** Weighted by 3 (Low), 4 (Average), or 6 (High).
> - **External Outputs (EO):** Weighted by 4 (Low), 5 (Average), or 7 (High).
> - **External Inquiries (EQ):** Weighted by 3 (Low), 4 (Average), or 6 (High).
> - **Internal Logical Files (ILF):** Weighted by 7 (Low), 10 (Average), or 15 (High).
> - **External Interface Files (EIF):** Weighted by 5 (Low), 7 (Average), or 10 (High).
> - $\mathrm{UFP} = \sum_{i=1}^{5} \sum_{j=1}^{3} W_{ij} \cdot Z_{ij}$.
> - **2. Phase 2: Computing Complexity Adjustment Factor (CAF):** The system is rated against 14 General System Characteristics (F_1 to F_{14}, including data communications, distributed processing, performance, transaction rate, online updates, operational ease, etc.) on a scale from 0 (no influence) to 5 (essential influence).
> - $\mathrm{CAF} = 0.65 + 0.01 \times \sum_{i=1}^{14} F_i$, providing a scaling range between 0.65 ($\sum F_i = 0$) and 1.35 ($\sum F_i = 70$), allowing an adjustment of $\pm 35\%$.
> - **Final Function Points:** $\mathrm{FP} = \mathrm{UFP} \times \mathrm{CAF}$.

Worked Example:

Consider an inventory management module with the following characteristics:

- Number of External Inputs = 10 (all of Average complexity: $10 \times 4 = 40$)
- Number of External Outputs = 5 (all of Average complexity: $5 \times 5 = 25$)
- Number of External Inquiries = 4 (all of Average complexity: $4 \times 4 = 16$)
- Number of Internal Logical Files = 2 (all of Average complexity: $2 \times 10 = 20$)
- Number of External Interface Files = 1 (all of Average complexity: $1 \times 7 = 7$)
- $\mathrm{UFP} = 40 + 25 + 16 + 20 + 7 = 108$.

Assume the sum of all 14 General System Characteristics ratings is evaluated to be $\sum F_i = 35$ (moderate influence):

- $\mathrm{CAF} = 0.65 + 0.01 \times 35 = 0.65 + 0.35 = 1.00$.
- $\mathrm{FP} = \mathrm{UFP} \times \mathrm{CAF} = 108 \times 1.00 = 108\ \mathrm{FP}$.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.3.2, pp. 145–148; Companion Slides 11–24.

Exercise 4.15. Types of COCOMO Modes & Phase-Wise Distribution of Effort

Discuss various types of COCOMO mode. Explain the phase wise distribution of effort.

Types of COCOMO Modes: Barry Boehm's Constructive Cost Model (COCOMO '81) categorizes software projects into three distinct operating modes based on project complexity, environmental constraints, and team experience:

- **1. Organic Mode:** Small, straightforward software projects (typically $\leq 50\ \mathrm{KLOC}$) developed by small, experienced teams working in familiar in-house environments with relaxed, flexible specifications. Examples include payroll systems, inventory tracking, and standard business utilities.
- **2. Semidetached Mode:** Intermediate software projects (typically $50 - 300\ \mathrm{KLOC}$) characterized by mixed team experience (some members experienced, others unfamiliar with the domain) and moderately stringent operational constraints. Examples include transaction processing systems, database management engines, and new operating system utilities.
- **3. Embedded Mode:** Highly complex software projects (typically $> 300\ \mathrm{KLOC}$, or any size operating within rigid, unforgiving operational constraints) developed under tight hardware, software, and regulatory regulations where changes are costly and interfaces are inflexible. Examples include air traffic control, avionics, medical device controllers, and defense command-and-control software.

Phase-Wise Effort and Schedule Distribution (Table 4.7): Table 4.7 (p. 159) gives separate effort (μ_p) and schedule (τ_p) fractions for five phases: Plan & Requirement, System Design, Detail Design, Module Code & Test, and Integration & Test. For the **Organic Small,** $S \approx 2$ row, the values are:

- **Effort fractions (μ_p):** 0.06, 0.16, 0.26, 0.42, 0.16 — total **1.06 (106%)**.

- **Schedule fractions (τ_p):** 0.10, 0.19, 0.24, 0.39, 0.18 — total **1.10 (110%)**.

The Plan & Requirement phase is conceptual and additional to estimated effort and time; actual distribution may start from System Design. The displayed fractions total 106% effort and 110% schedule. No cause is stated for these totals, so preserve them when reporting the table; identify any normalization explicitly when a calculation requires a 100% allocation.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.4.3, printed p. 159 (Table 4.7); Companion Slides 62–63.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- صف Organic Small في الجدول اسمه $S \approx 2$؛ لا تحوّل هذا الرمز إلى حدّ عام مثل ≤ 50 KLOC من دون سند مستقل.
- لاحظ أن مرحلة البرمجة وفحص الوحدات (Unit Test & Coding) تستحوذ دائماً على النسبة الكبرى من الجهد (~40%).

Exercise 4.22. COCOMO II in Detail & Project Categories

Explain the COCOMO-II in detail. What types of categories of projects are identified?

COCOMO II Overview: Developed by Barry Boehm at the University of Southern California (USC) in the 1990s, COCOMO II modernizes the 1981 model to accommodate modern software development paradigms: object-oriented frameworks, commercial-off-the-shelf (COTS) components, rapid application development (RAD), and automated code generators. It replaces the single-model assumption with a multi-stage framework that matches estimation fidelity to the information available across the life cycle.

Categories of Projects Identified in COCOMO II (Slide 73 / Textbook p. 162):

- **1. End-User Programming:** Applications generated directly by end users using visual scripting, spreadsheet macros, and query builders (e.g., automated report generators).
- **2. Application Composition / Generators:** Projects constructed rapidly using visual pre-built components, graphical user interface (GUI) builders, and specialized database application generators.
- **3. System Integration:** Large-scale systems built primarily by integrating existing heterogeneous subsystems, legacy components, and commercial COTS packages into an integrated enterprise solution.
- **4. Infrastructure Sector:** Deep systems software requiring extreme performance, high reliability, and tight hardware-software coupling, such as operating systems, database management system kernels, network protocol stacks, and real-time compilers.

The Three Estimation Stages of COCOMO II:

- **Stage 1: Application Composition Model:** Used during prototyping; sizing is in **Object Points (NOP)**, and effort is estimated via productivity tables: $\mathrm{PM} = \frac{\mathrm{NOP}}{\mathrm{PROD}}$.

- **Stage 2: Early Design Model:** Used before architecture is locked; employs 5 scale factors (SF_j) to determine the exponent $B = 0.91 + 0.01 \sum SF_j$ and adjusts nominal effort using 7 early design cost drivers (RCPX, RUSE, PDIF, PERS, PREX, FCIL, SCED).
- **Stage 3: Post-Architecture Model:** Used after the life-cycle architecture is baselined; it uses the 5 scale factors and 17 post-architecture cost drivers to estimate effort and schedule.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.5, pp. 161–171; Companion Slides 73–86.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- ميز بين قطاعات المشاريع الأربعة (End-user, Application Composition, System Integration, Infrastructure) وبين المراحل التقديرية الثلاث (Application Composition, Early Design, Post-Architecture).

Exercise 4.23. The Infrastructure Sector of COCOMO II

Discuss the Infrastructure Sector of COCOMO-II.

The Infrastructure Sector is one of the four COCOMO II application sectors. It covers systems software such as operating systems, database systems, compilers, and related infrastructure.

It identifies the type of software being estimated, not an estimation stage. Choose a COCOMO II stage according to the project information available; the sector label alone does not determine a stage or cost-driver rating.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.5, p. 162; Companion Slides 73, 198.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- لا تخلط بين **Infrastructure Sector** وبين مرحلة التقدير: القطاع يصف نوع البرنامج، والمرحلة تُختار حسب المعلومات المتاحة. جدول المراحل في الفصل يدرج البنية التحتية ضمن القطاعات التي يمكن أن تخدمها أكثر من مرحلة؛ لذلك لا يفرض اسم القطاع وحده Post-Architecture.

Exercise 4.26. The Putnam Resource Allocation Model & Limitations

Explain the Putnam resource allocation model. What are the limitations of this model?

The Putnam Resource Allocation Model: The model represents project staffing over time with a Rayleigh curve:

$$m(t) = 2Kat \cdot e^{-at^2} = \frac{K}{t_d^2} \cdot t \cdot e^{-\frac{t^2}{2t_d^2}}$$

Where:

- $m(t)$ is the manpower utilization rate (persons/year) at elapsed project time t.
- K is the total life-cycle effort (person-years), covering development and maintenance.
- t_d is the time of peak manpower utilization, which coincides with system deployment/release.
- $a = \frac{1}{2t_d^2}$ is the shape parameter.
- **Peak Manning:** At $t = t_d$, maximum staffing is $m_0 = \frac{K}{t_d\sqrt{e}} \approx 0.6065\frac{K}{t_d}$.
- **The Software Equation:** Relates product size S to effort K and schedule t_d: $S = C_k \cdot K^{1/3} \cdot t_d^{4/3}$ (or $K \propto t_d^{-4}$).

Model assumptions and limitations:

- **1. Schedule sensitivity:** With size and technology factor held fixed, the model gives $K \propto t_d^{-4}$. A 50% duration reduction therefore multiplies modeled effort by 16. This is a relationship within the model, not a universal forecast for every project.
- **2. Staffing-shape assumption:** The model represents staffing as a smooth Rayleigh curve; an actual project may use a different staffing profile.
- **3. Calibration context:** Use the model parameters with awareness of their calibration context; the model itself does not define a universal minimum project size.
- **4. Technology factor:** Estimates depend on the selected technology factor C_k; state its value and units when applying the software equation.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.6, printed pp. 182–191; Companion Slides 123–140.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

ملاحظة الطالب والفخ الامتحاني:

- معادلة ذروة التوظيف $m_0 = \frac{K}{t_d\sqrt{e}}$ تحتوي جذراً تربيعياً على e؛ لا تكتب e مفردة.
- المنحنى والارتباط بين الوقت والجهد افتراضات للنموذج؛ قارنها ببيانات المشروع ولا تعامل النتائج كقواعد عامة لكل مشروع.

Exercise 4.33. Risk Management Activities & Risk Prioritization

What are risk management activities? Is it possible to prioritize the risk?

Risk Management Activities: Software risk management has two main activities, each with three sub-activities:

- **1. Risk Assessment:**

- **Risk Identification:** Identifying potential risk areas, using a checklist of common areas or an organizational database of previously identified risks.
- **Risk Analysis:** Examining how project outcomes might change when risk input variables change.
- **Risk Prioritization:** Estimating Risk Exposure for each risk and giving attention first to the largest exposures.
- **2. Risk Control:**
- **Risk Management Planning:** Preparing a plan for dealing with each significant risk and recording the decision.
- **Risk Resolution:** Executing the plan for dealing with a risk.
- **Risk Monitoring:** Monitoring risks during the project.

Is it Possible to Prioritize Risk? (Yes — Quantitative Prioritization): Yes. Estimate the Risk Exposure (RE) for each risk and prioritize the risks with the largest exposure:

$$\mathrm{RE} = P \times L$$

Where:

- P is the probability of occurrence ($0 < P < 1$).
- L is the potential severity of loss (measured in financial cost, schedule delay days, or person-months).
- **Prioritization Rule:** Estimate the Risk Exposure of each risk and give attention first to risks with the highest exposure.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.7.3, pp. 196–198; Companion Slides 190–193.

Exercise 4.34. Risk Exposure & Techniques to Control Each Risk

What is risk exposure? What techniques can be used to control each risk?

Risk Exposure (RE): Risk Exposure is the product of the probability of incurring a loss due to a risk and the potential magnitude of that loss:

$$\mathrm{RE} = P \times L$$

Risk-Control Activities and a Source Example:

- **Risk Avoidance:** Avoid undertaking a risky project, or choose a proven technology instead of a cutting-edge one.
- **Risk Management Planning:** Prepare and record a plan for each significant risk.
- **Risk Resolution:** Carry out the selected plan.
- **Risk Monitoring:** Track the risk as the project proceeds.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.7.3, pp. 196–198; Companion Slides 190–193.

📝 **ملاحظة أكاديمية هامة (Academic Note)**

ملاحظة الطالب والفخ الامتحاني:

- لا تخلط بين تجنب الخطر وبين خطة التعامل معه: التجنب يغيّر القرار أو التقنية لتفاديه، أما التخطيط والتحكم فيتعاملان مع خطر سيبقى ضمن المشروع.

Section 2: Curriculum Gap-Closing Synthesis Questions (6 Items)

Synthesis Q2.1. Lines of Code (LOC) Counting Conventions & Sizing Invariants

Explain the Lines of Code (LOC) metric. What are the distinctions between physical and logical lines, and what are the main limitations of LOC?

LOC measures software size by counting source-program text. One LOC convention counts a line of program text that is neither a comment nor blank; it counts as one LOC regardless of how many statements or statement fragments appear on that line.

- **Physical-line counting** counts eligible program-text lines.
- **Logical-statement counting** counts source statements; one physical line may contain more than one statement, and one statement may span lines.

State the counting rule being used because LOC conventions can differ.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.3.1, pp. 143–145; Companion Slides 7–8. [Curriculum Invariant]

Synthesis Q2.2. Static Cost Estimation Models: Single-Variable vs. Multivariable

Contrast static single-variable cost estimation models with static multivariable models, providing standard empirical examples of each.

Static Single-Variable Models: Static single-variable models estimate effort or duration from one predictor, commonly software size (L in KLOC):

$$E = a \cdot L^b \quad \text{and} \quad D = c \cdot L^d$$

- **Empirical Examples:**
- **Walston-Felix (IBM):** $E = 5.2 \cdot L^{0.91}\ \text{PM}, \quad D = 4.1 \cdot L^{0.36}\ \text{Months}.$
- **SEL Model (NASA/Maryland):** $E = 1.4 \cdot L^{0.93}\ \text{PM}, \quad D = 4.6 \cdot L^{0.26}\ \text{Months}.$
- **Limitation:** Assumes all other project factors (staff experience, tool support, complexity) remain constant, which fails when projects deviate from the organization's historical baseline.

Static Multivariable Models: Static multivariable models acknowledge that software cost depends on multiple simultaneous environmental and technical parameters. They take the general algebraic form:

$$E = a_0 + a_1x_1 + a_2x_2 + \cdots + a_nx_n \quad \text{or} \quad E = a_0 \cdot \prod_{i=1}^{n} x_i^{a_i}$$

- **Empirical Example (Walston-Felix Multivariable Extension):** Walston–Felix also defines a productivity index using 29 project variables: $I = \sum_{i=1}^{29} W_i X_i$, where W_i is a factor weight and X_i

is rated -1, 0, or $+1$ according to its effect on productivity. The index is used with the productivity-index chart.

- **Distinction:** Unlike a single-variable model, the estimate depends on multiple stated project variables.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.3.2, printed p. 151; §4.4, pp. 150–152; Companion Slides 34–40. [Curriculum Invariant]

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- النماذج أحادية المتغير (Single-variable) تستند لمتغير واحد هو الحجم (L). مثال الفصل هو مؤشر إنتاجية Felix–Walston ذي 29 متغيراً؛ لا تخلط بين عدد عوامل المؤشر وعدد متغيرات نموذج آخر.

Synthesis Q2.3. Structural Levels of COCOMO '81: Basic, Intermediate, and Detailed

Explain the three structural tiers of Barry Boehm's COCOMO '81 model, highlighting how cost drivers and phase distributions are incorporated.

The Three Tiers of COCOMO '81: Barry Boehm formulated COCOMO '81 in three progressive levels of mathematical detail:

- **1. Basic COCOMO:**
- **Mechanism:** Computes nominal effort and development time strictly as a power function of software size (KLOC): $E = a_b(\mathrm{KLOC})^{b_b}$ and $D = c_b(E)^{d_b}$.
- **Application:** Useful for quick, rough, early-order-of-magnitude estimates where project-specific environmental details are still unknown.
- **2. Intermediate COCOMO:**
- **Mechanism:** Refines nominal effort by multiplying it by an Effort Adjustment Factor (EAF) derived from **15 cost drivers (effort multipliers)** rated across four domains: Product attributes (RELY, DATA, CPLX), Computer attributes (TIME, STOR, VIRT, TURN), Personnel attributes (ACAP, AEXP, PCAP, VEXP, LEXP), and Project attributes (MODP, TOOL, SCED).
- **Formula:** $E = a_i(\mathrm{KLOC})^{b_i} \times \prod_{j=1}^{15} EM_j$.
- **3. Detailed COCOMO:**
- **Mechanism:** Acknowledges that cost drivers do not impact all life cycle phases uniformly. It uses phase-sensitive multipliers so that effort and schedule adjustments can vary by lifecycle phase. Exact revised phase shares require the corresponding phase-level multipliers.
- **Phase Breakdown:** Detailed COCOMO uses **phase-sensitive effort multipliers** allocated across the five Table 4.7 phases: Plan & Requirement, System Design, Detail Design, Module Code & Test, and Integration & Test.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.4.1–§4.4.3, pp. 152–160; Companion Slides 41–65. [Curriculum Invariant]

Synthesis Q2.4. The Three Estimation Stages of COCOMO II

Explain the three estimation stages of COCOMO II and discuss the criteria for selecting between them.

The Three Stages of COCOMO II: COCOMO II provides three estimation models that align with the increasing granularity of system knowledge as a project moves from initial conception to deployment:

- **1. Stage 1: Application Composition Model:**
- **Lifecycle Phase:** Early prototyping and exploration phase.
- **Sizing Metric: Object Points (NOP)**, evaluating UI screens, printable reports, and 3GL component modules.
- **Effort Calculation:** $\mathrm{PM} = \frac{\mathrm{NOP}}{\mathrm{PROD}}$, where productivity rate (PROD) is selected from empirical developer capability tables.
- **2. Stage 2: Early Design Model:**
- **Lifecycle Phase:** Architecture formulation phase (before detailed design choices are frozen).
- **Sizing Metric:** Unadjusted Function Points (UFP) converted to KLOC.
- **Effort Calculation:** $\mathrm{PM}_{\mathrm{nominal}} = 2.5 \times (\mathrm{Size})^{B}$, where exponent B is determined by **5 Scale Factors** ($B = 0.91 + 0.01 \sum SF_j$), adjusted by **7 Early Design Effort Multipliers** (RCPX, RUSE, PDIF, PERS, PREX, FCIL, SCED).
- **3. Stage 3: Post-Architecture Model:**
- **Lifecycle Phase:** Detailed design, implementation, and integration phase.
- **Sizing Metric:** Source Lines of Code (SLOC) or Function Points, adjusted for code reuse, component breakage, and automatic generation.
- **Effort Calculation:** Uses the same 5 Scale Factors but applies **17 detailed Post-Architecture Cost Drivers** (covering personnel, platform, product, and project parameters).

Selection Criteria: Model choice follows the **availability of project information**. Stage 1 fits application composition and prototyping; Stage 2 is used when early sizing and aggregated drivers can be assessed; Stage 3 needs more detailed architecture and cost-driver information.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.5, pp. 161–171; Companion Slides 73–86. [Curriculum Invariant]

Synthesis Q2.5. Putnam Model Foundations: Norden's Rayleigh Curve & Development Sub-Cycle

Explain how Norden's Rayleigh curve forms the foundation of Putnam's resource model, and distinguish the full project lifecycle from the development sub-cycle.

Norden's Rayleigh Curve Foundation: The model represents manpower as a Rayleigh curve: staffing rises to a peak and then declines. Putnam uses the curve as the basis of the resource-allocation model.

$$m(t) = 2Kat \cdot e^{-at^2} = \frac{K}{t_d^2} \cdot t \cdot e^{-\frac{t^2}{2t_d^2}}$$

- **Peak Staffing Rate (m_0):** Occurs exactly at elapsed time $t = t_d$: $m_0 = \frac{K}{t_d\sqrt{e}} \approx 0.6065\frac{K}{t_d}$.
- **Difficulty Metric (D):** $D = \frac{K}{t_d^2}$.

Distinction Between Full Lifecycle and Development Sub-Cycle:

- **1. Full Project Lifecycle (K, t_d):**
- K is the effort parameter for the whole-project Rayleigh curve; distinguish it from the development-sub-cycle effort K_d.
- t_d is the time when development ends and system deployment occurs.
- **2. Effort accumulated by release:**
- Integrating the full-project Rayleigh curve from 0 to t_d gives $E(t_d) = K(1 - e^{-1/2}) \approx 0.3935K$. This is cumulative effort by release on that curve; it is distinct from the development-sub-cycle total K_d.
- **3. Development Sub-Cycle (K_d, t_{od}):**
- Total project effort and development-sub-cycle effort are related by $K = 6K_d$, so $K_d = K/6$.
- For the development curve, the peak time is $t_{od} = t_d/\sqrt{6}$, and peak staffing is derived from that curve's own parameters.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.6, printed pp. 182–191; Companion Slides 123–140, 151. [Curriculum Invariant]

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- لا تخلط بين الجهد التراكمي حتى t_d على منحنى المشروع ($\approx 0.3935K$) وبين جهد دورة التطوير K_d؛ أغروال يذكر العلاقة $K = 6K_d$.
- للمنحنى الكامل تبلغ الذروة عند t_d؛ أما منحنى التطوير فله زمن ذروة مستقل $t_{od} = t_d/\sqrt{6}$.

Synthesis Q2.6. The Five Typical Software-Risk Categories

List the five typical software-risk categories presented in the course material and give one representative factor for each.

The five categories and representative factors are: - **Dependencies on outside agencies or factors:** Availability of trained, experienced people. - **Requirement issues:** Unclear or changing requirements. - **Management issues:** Inadequate planning or poor communication. - **Lack of knowledge:** Inadequate training or insufficient application-domain experience. - **Other risk categories:** Turnover of essential personnel or unavailable testing facilities.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §4.7.2, pp. 195–196; Companion Slides 184–189.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- لا تخلط بين هذه الفئات الخمس وبين تصنيفات مخاطر أخرى من مراجع مختلفة؛ احفظ أسماء الفئات وأمثلتها كما وردت في محاضرة هذا الفصل.